



Bus Reservation System

BusVoyager — Full-Stack Web Application

Project Report

A Java Full-Stack Project using Spring Boot & MongoDB

Submitted By: Janiish R

DEPARTMENT: CSBS

Table of Contents

1. Introduction

- 1.1 Problem Statement
- 1.2 Objectives
- 1.3 Scope

2. Literature Review

3. System Requirements

- 3.1 Hardware Requirements
- 3.2 Software Requirements
- 3.3 Functional Requirements
- 3.4 Non-Functional Requirements

4. System Architecture & Design

- 4.1 High-Level Architecture
- 4.2 Technology Stack
- 4.3 Project Structure
- 4.4 Database Design
- 4.5 REST API Design

5. Implementation

- 5.1 Backend — Entity Layer
- 5.2 Backend — Repository Layer
- 5.3 Backend — Service Layer
- 5.4 Backend — Controller Layer
- 5.5 Backend — DTOs
- 5.6 Backend — Configuration & Exception Handling
- 5.7 Frontend — User Interface

6. User Interface Screenshots

7. Testing

8. Future Enhancements

9. Conclusion

10. References

Appendix — Complete Source Code

1. Introduction

The **Bus Reservation System (BusVoyager)** is a full-stack web application designed to simplify the process of searching for buses, selecting seats, and booking tickets online. The system replaces traditional manual ticket-booking counters with a modern, user-friendly digital interface backed by a robust REST API.

Built using **Spring Boot 3.4** on the backend and **vanilla HTML/CSS/JavaScript** with **Tailwind CSS** on the frontend, the application demonstrates enterprise-grade development practices including layered architecture, DTO pattern, global exception handling, and MongoDB as the persistence store.

1.1 Problem Statement

India's intercity bus network serves millions of passengers daily. However, many operators still rely on manual or fragmented systems for seat allocation, leading to:

- Long queues and delayed departures at bus terminals.
- Overbooking or under-utilisation of bus capacity.
- Lack of real-time seat availability information for passengers.
- No centralised record of bookings for auditing or analytics.

A digital reservation system addresses every one of these pain points while improving customer satisfaction and operational efficiency.

1.2 Objectives

1. Develop a RESTful backend API using **Spring Boot** and **MongoDB** to manage buses, routes, and bookings.
2. Create an intuitive, responsive frontend for passengers to search buses, view seat maps, and book tickets.
3. Implement real-time seat availability tracking with conflict detection to prevent double-booking.
4. Apply clean software engineering practices: layered architecture, DTO pattern, centralised error handling, and CORS configuration.
5. Provide seed data and a database schema for easy project setup and demonstration.

1.3 Scope

The current version of the system covers:

- Bus search by origin, destination, and travel date.
- Interactive seat selection with a visual 2+2 seat map.
- Booking creation with passenger details and fare calculation.
- Booking confirmation with a printable ticket.
- City listing fetched dynamically from the database.

Out of scope (for this version): user authentication, payment gateway integration, admin panel, and cancellation/refund workflows.

2. Literature Review

Online reservation systems have been extensively studied in the fields of e-commerce, transportation, and enterprise application design. This chapter surveys the key technologies and architectural patterns adopted in this project.

2.1 Spring Boot & REST API Design

Spring Boot, part of the Spring ecosystem, provides convention-over-configuration for building production-ready applications. Its embedded Tomcat server, auto-configuration, and starter dependencies drastically reduce boilerplate. RESTful API design—mapping HTTP verbs (GET, POST, PUT, DELETE) to CRUD operations—has become the de-facto standard for client-server communication in modern web development (Fielding, 2000).

2.2 NoSQL Databases — MongoDB

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents. For applications with rapidly evolving schemas or nested data (e.g., seat number arrays within bookings), MongoDB provides superior flexibility over rigid relational schemas. Spring Data MongoDB offers repository abstractions that eliminate hand-written CRUD queries (MongoDB Inc., 2024).

2.3 Frontend Technologies

Modern frontends leverage utility-first CSS frameworks such as **Tailwind CSS** for rapid UI development. Combined with the Fetch API for asynchronous HTTP calls, single-page experiences can be built without heavy framework overhead (React, Angular). The BusVoyager frontend adopts this lightweight approach while delivering a premium glassmorphism aesthetic.

2.4 Layered Architecture & DTO Pattern

The three-tier architecture (Controller → Service → Repository) separates concerns, improves testability, and allows independent evolution of layers. Data Transfer Objects (DTOs) decouple the API contract from internal entity models, preventing unintended data exposure and offering flexibility for API versioning (Fowler, 2002).

3. System Requirements

3.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 / AMD Ryzen 3 or above
RAM	4 GB (8 GB recommended)
Storage	500 MB free disk space
Network	Internet connectivity for dependency resolution

3.2 Software Requirements

Software	Version	Purpose
JDK	25	Java runtime & compilation
Apache Maven	3.9+	Build & dependency management
MongoDB	7.0+	NoSQL database
Spring Boot	3.4.1	Backend framework
Web Browser	Chrome / Edge / Firefox (latest)	Frontend rendering
IDE (optional)	IntelliJ IDEA / VS Code	Development environment

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	Search buses by origin, destination, and date	High
FR-02	Display real-time seat availability for a bus	High
FR-03	Allow passengers to select up to 6 seats	High
FR-04	Create a booking with passenger details	High
FR-05	Display booking confirmation with ticket details	High
FR-06	Prevent double-booking of seats	High

FR-07	List all serviced cities dynamically	Medium
FR-08	Filter buses by operating days	Medium

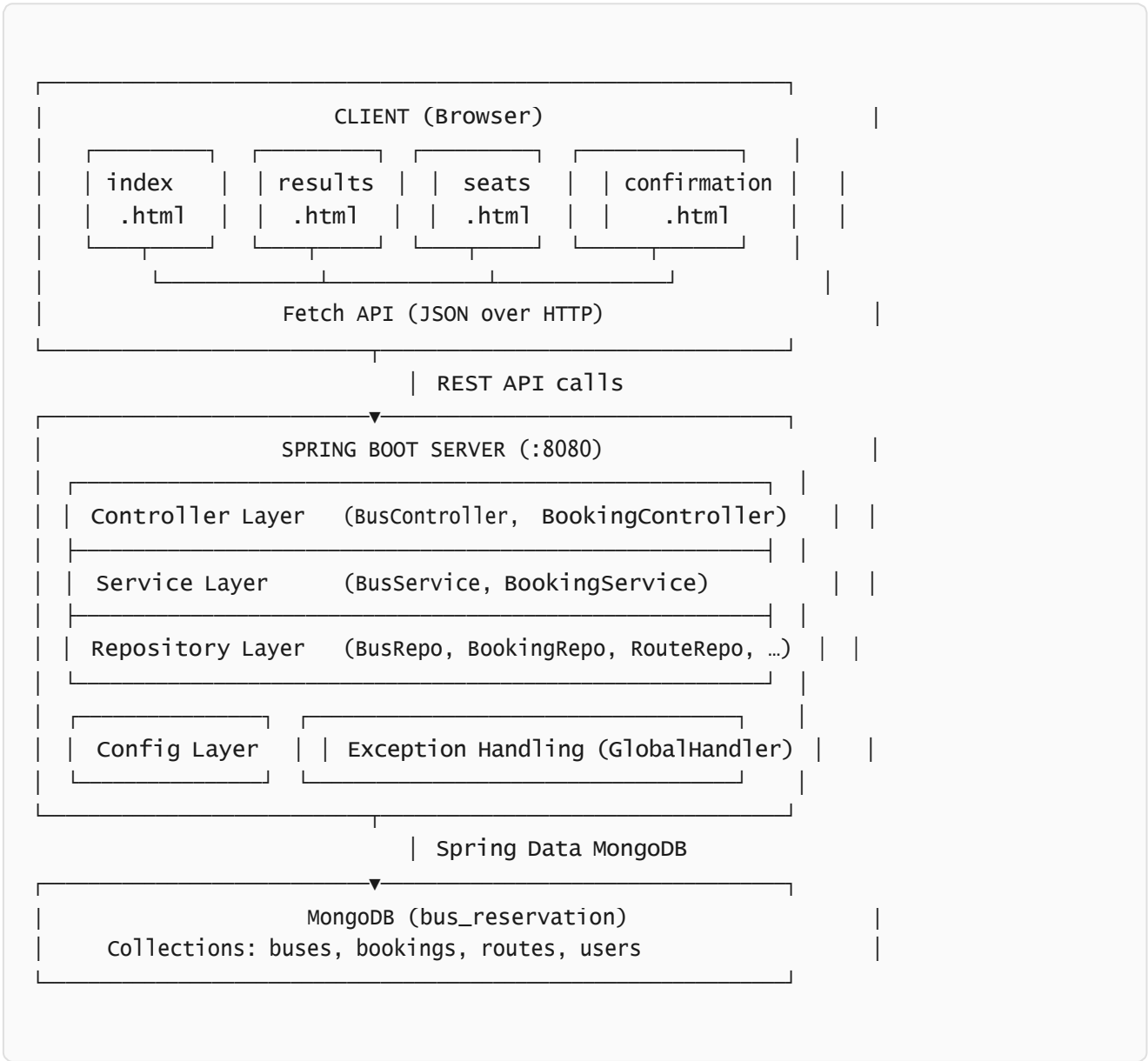
3.4 Non-Functional Requirements

ID	Requirement
NFR-01	API response time $\leq$ 500 ms under normal load
NFR-02	Frontend pages load within 2 seconds on broadband
NFR-03	Responsive design supporting mobile, tablet, and desktop
NFR-04	Clean, layered code architecture for maintainability
NFR-05	Cross-Origin Resource Sharing (CORS) enabled for all origins

4. System Architecture & Design

4.1 High-Level Architecture

The application follows a **Client-Server** model with a clear separation between the frontend (HTML/JS) and the backend (Spring Boot REST API). The backend itself is organised in a **three-tier layered architecture**.



4.2 Technology Stack

Layer	Technology	Role
-------	------------	------

Frontend	HTML5, Tailwind CSS, JavaScript (ES6+)	User interface, DOM manipulation, API calls
Backend Framework	Spring Boot 3.4.1	REST API, dependency injection, auto-configuration
Data Access	Spring Data MongoDB	Repository abstraction over MongoDB
Database	MongoDB 7.0+	NoSQL document store
Build Tool	Apache Maven	Dependency management & build lifecycle
Language	Java 25	Backend programming language
Utility Library	Lombok	Boilerplate reduction (getters, setters, constructors)
Validation	Spring Boot Starter Validation	Bean validation

4.3 Project Structure

```

Project/
├─ bus-reservation-backend/
│   ├─ pom.xml
│   └─ src/main/java/com/busreserve/
│       ├─ BusReservationApplication.java    ← Entry Point
│       ├─ config/
│       │   └─ CorsConfig.java              ← CORS Configuration
│       ├─ controller/
│       │   ├─ BusController.java           ← Bus search & cities API
│       │   └─ BookingController.java       ← Booking create & fetch API
│       ├─ dto/
│       │   ├─ BookingRequest.java
│       │   ├─ BookingResponse.java
│       │   ├─ BusSearchResponse.java
│       │   └─ SeatAvailabilityResponse.java
│       ├─ entity/
│       │   ├─ Bus.java
│       │   ├─ Booking.java
│       │   ├─ Route.java
│       │   └─ User.java
│       ├─ exception/
│       │   └─ GlobalExceptionHandler.java
│       └─ repository/
│           ├─ BusRepository.java
│           ├─ BookingRepository.java
│           └─ RouteRepository.java

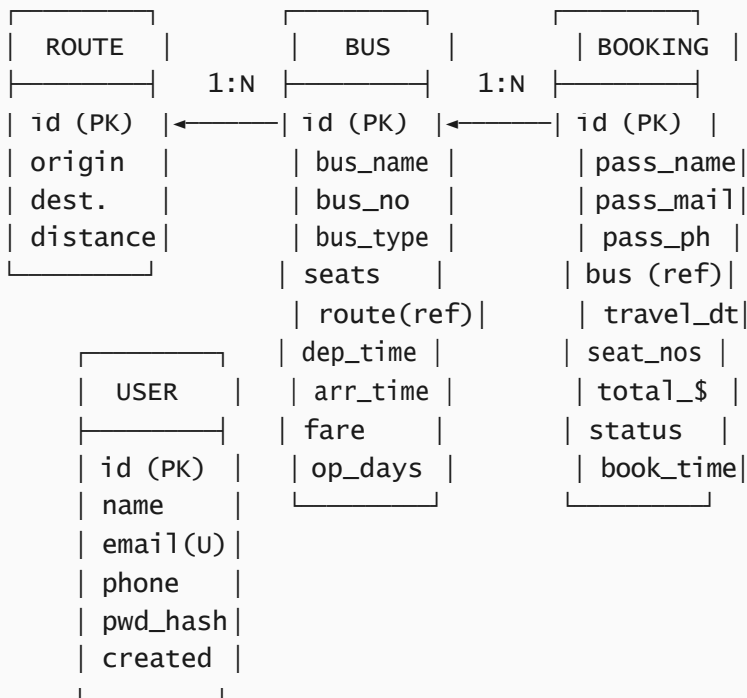
```

```
|      |   └─ UserRepository.java
|      └─ service/
|          └─ BusService.java
|              └─ BookingService.java
└─ database/
    └─ schema.sql          ← Schema & Seed Data
└─ frontend/
    └─ index.html          ← Home / Search Page
    └─ results.html        ← Search Results Page
    └─ seats.html          ← Seat Selection Page
    └─ confirmation.html   ← Booking Confirmation
```

4.4 Database Design

The application uses four MongoDB collections. The equivalent relational schema (provided in `schema.sql`) is shown below as an ER model reference:

4.4.1 Entity-Relationship Diagram (Textual)



4.4.2 Collection Descriptions

Collection	Key Fields	Description
routes	id, origin, destination, distance_km	Stores city-to-city routes with distance information
buses	id, bus_name, bus_number, bus_type, total_seats, route (DBRef), departure_time, arrival_time, fare, operating_days	Bus details linked to a route via document reference
bookings	id, passenger_name/email/phone, bus (DBRef), travel_date, seat_numbers (List), total_fare, status, booking_time	Individual bookings with seat list and fare
users	id, name, email (unique indexed), phone, password_hash, created_at	User accounts (reserved for future authentication)

4.4.3 Seed Data

The `schema.sql` file provides seed data including:

- **3 users** — Rahul Sharma, Priya Patel, Arjun Kumar
- **8 routes** — 4 bidirectional pairs: Delhi↔Mumbai, Bangalore↔Chennai, Hyderabad↔Pune, Jaipur↔Delhi
- **12 buses** — Across multiple routes with AC, Non-AC, Sleeper, and Semi-Sleeper types
- **3 sample bookings** — Pre-booked seats for demonstration

4.5 REST API Design

Method	Endpoint	Description	Request	Response
GET	/api/buses/search	Search buses by route & date	Query: from, to, date	List<BusSearchResponse>
GET	/api/buses/{id}/seats	Get seat availability	Path: id, Query: date	SeatAvailabilityResponse
GET	/api/cities	List all serviced cities	—	List<String>
POST	/api/bookings	Create a new booking	Body: BookingRequest (JSON)	BookingResponse
GET	/api/bookings/{id}	Retrieve booking details	Path: id	BookingResponse

5. Implementation

This chapter walks through the key source code files, organised by architectural layer.

5.1 Backend — Entity Layer

Entities are MongoDB document models annotated with `@Document`. Lombok annotations eliminate boilerplate getters, setters, and constructors.

5.1.1 Bus.java

```
@Document(collection = "buses")
@Data @NoArgsConstructor @AllArgsConstructor
public class Bus {
    @Id
    private String id;

    @Field("bus_name")
    private String busName;

    @Field("bus_number")
    private String busNumber;

    @Field("bus_type")
    private String busType;

    @Field("total_seats")
    private Integer totalSeats = 40;

    @DBRef
    private Route route;

    @Field("departure_time")
    private LocalTime departureTime;

    @Field("arrival_time")
    private LocalTime arrivalTime;

    private BigDecimal fare;

    @Field("operating_days")
    private String operatingDays;
}
```

5.1.2 Booking.java

```

@Document(collection = "bookings")
@Data @NoArgsConstructor @AllArgsConstructor
public class Booking {
    @Id
    private String id;

    @Field("passenger_name") private String passengerName;
    @Field("passenger_email") private String passengerEmail;
    @Field("passenger_phone") private String passengerPhone;

    @DBRef
    private Bus bus;

    @Field("travel_date") private LocalDate travelDate;
    @Field("seat_numbers") private List<Integer> seatNumbers;
    @Field("total_fare") private BigDecimal totalFare;
    private String status = "CONFIRMED";
    @Field("booking_time") private LocalDateTime bookingTime = LocalDateTime.now();
}

```

5.1.3 Route.java

```

@Document(collection = "routes")
@Data @NoArgsConstructor @AllArgsConstructor
public class Route {
    @Id
    private String id;
    private String origin;
    private String destination;
    @Field("distance_km")
    private Integer distanceKm;
}

```

5.1.4 User.java

```

@Document(collection = "users")
@Data @NoArgsConstructor @AllArgsConstructor
public class User {
    @Id
    private String id;
    private String name;
    @Indexed(unique = true)
    private String email;
    private String phone;
    @Field("password_hash")
    private String passwordHash;
    @Field("created_at")
}

```

```
private LocalDateTime createdAt = LocalDateTime.now();  
}
```


5.2 Backend — Repository Layer

Spring Data MongoDB repositories provide zero-implementation CRUD and custom query methods through method name conventions.

5.2.1 BusRepository.java

```
@Repository
public interface BusRepository extends MongoRepository<Bus, String> {
    List<Bus> findByRoute(Route route);
}
```

5.2.2 BookingRepository.java

```
@Repository
public interface BookingRepository extends MongoRepository<Booking, String> {
    List<Booking> findByBusIdAndTravelDateAndStatus(
        String busId, LocalDate travelDate, String status);
}
```

5.2.3 RouteRepository.java

```
@Repository
public interface RouteRepository extends MongoRepository<Route, String> {
    List<Route> findByOriginIgnoreCaseAndDestinationIgnoreCase(
        String origin, String destination);
}
```

5.2.4 UserRepository.java

```
@Repository
public interface UserRepository extends MongoRepository<User, String> {
}
```

5.3 Backend — Service Layer

Services encapsulate all business logic. They interact with repositories and return DTOs to controllers.

5.3.1 BusService.java

```
@Service
@RequiredArgsConstructor
public class BusService {
```

```

private final BusRepository busRepository;
private final BookingRepository bookingRepository;
private final RouteRepository routeRepository;

public List<BusSearchResponse> searchBuses(String from, String to, String dateStr)
{
    LocalDate travelDate = LocalDate.parse(dateStr);
    String day = travelDate.getDayOfWeek().name().substring(0, 3);

    List<Route> routes = routeRepository
        .findByOriginIgnoreCaseAndDestinationIgnoreCase(from, to);
    if (routes.isEmpty()) return Collections.emptyList();

    return busRepository.findByRoute(routes.get(0)).stream()
        .filter(bus -> bus.getOperatingDays() != null
            && bus.getOperatingDays().contains(day))
        .map(bus -> {
            int booked = countBookedSeats(bus.getId(), travelDate);
            return new BusSearchResponse(bus.getId(), bus.getBusName(),
                bus.getBusNumber(), bus.getBusType(), bus.getTotalSeats(),
                bus.getDepartureTime(), bus.getArrivalTime(), bus.getFare(),
                bus.getRoute().getOrigin(), bus.getRoute().getDestination(),
                bus.getTotalSeats() - booked);
        }).collect(Collectors.toList());
}

public SeatAvailabilityResponse getSeatAvailability(String busId, String dateStr) {
    LocalDate travelDate = LocalDate.parse(dateStr);
    Bus bus = busRepository.findById(busId)
        .orElseThrow(() -> new RuntimeException("Bus not found"));

    List<Integer> bookedSeats = bookingRepository
        .findByBusIdAndTravelDateAndStatus(busId, travelDate, "CONFIRMED")
        .stream().filter(b -> b.getSeatNumbers() != null)
        .flatMap(b -> b.getSeatNumbers().stream())
        .distinct().collect(Collectors.toList());

    return new SeatAvailabilityResponse(
        bus.getId(), bus.getBusName(), bus.getTotalSeats(),
        bookedSeats, bus.getFare());
}

public List<String> getAllCities() {
    return routeRepository.findAll().stream()
        .flatMap(r -> List.of(r.getOrigin(), r.getDestination()).stream())
        .distinct().sorted().collect(Collectors.toList());
}

private int countBookedSeats(String busId, LocalDate date) {
    return bookingRepository
        .findByBusIdAndTravelDateAndStatus(busId, date, "CONFIRMED")

```

```
        .stream().filter(b -> b.getSeatNumbers() != null)
        .mapToInt(b -> b.getSeatNumbers().size()).sum();
    }
}
```

5.3.2 BookingService.java

```
@Service
@RequiredArgsConstructor
public class BookingService {
    private final BookingRepository bookingRepository;
    private final BusRepository busRepository;

    @Transactional
    public BookingResponse createBooking(BookingRequest request) {
        Bus bus = busRepository.findById(request.getBusId())
            .orElseThrow(() -> new RuntimeException("Bus not found"));

        LocalDate travelDate = LocalDate.parse(request.getTravelDate());
        Set<Integer> bookedSeats = getBookedSeatNumbers(bus.getId(), travelDate);

        for (Integer seatNo : request.getSeatNumbers()) {
            if (seatNo < 1 || seatNo > bus.getTotalSeats())
                throw new RuntimeException("Invalid seat number: " + seatNo);
            if (bookedSeats.contains(seatNo))
                throw new RuntimeException("Seat already booked: " + seatNo);
        }

        Booking booking = new Booking();
        booking.setPassengerName(request.getPassengerName());
        booking.setPassengerEmail(request.getPassengerEmail());
        booking.setPassengerPhone(request.getPassengerPhone());
        booking.setBus(bus);
        booking.setTravelDate(travelDate);
        booking.setTotalFare(bus.getFare()
            .multiply(BigDecimal.valueOf(request.getSeatNumbers().size())));
        booking.setSeatNumbers(request.getSeatNumbers());
        booking.setStatus("CONFIRMED");
        booking.setBookingTime(LocalDateTime.now());

        return mapToResponse(bookingRepository.save(booking));
    }

    public BookingResponse getBooking(String id) {
        return mapToResponse(bookingRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Booking not found")));
    }

    private Set<Integer> getBookedSeatNumbers(String busId, LocalDate date) {
        return bookingRepository
            .findByBusIdAndTravelDateAndStatus(busId, date, "CONFIRMED")
            .stream().filter(b -> b.getSeatNumbers() != null)
            .flatMap(b -> b.getSeatNumbers().stream())
            .collect(Collectors.toSet());
    }
}
```

```

private BookingResponse mapToResponse(Booking b) {
    return new BookingResponse(b.getId(), b.getPassengerName(),
        b.getPassengerEmail(), b.getPassengerPhone(),
        b.getBus().getBusName(), b.getBus().getBusNumber(),
        b.getBus().getRoute().getOrigin(),
        b.getBus().getRoute().getDestination(),
        b.getTravelDate(), b.getSeatNumbers(),
        b.getTotalFare(), b.getStatus(), b.getBookingTime());
}
}

```

5.4 Backend — Controller Layer

5.4.1 BusController.java

```

@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class BusController {
    private final BusService busService;

    @GetMapping("/buses/search")
    public ResponseEntity<List<BusSearchResponse>> searchBuses(
        @RequestParam("from") String from,
        @RequestParam("to") String to,
        @RequestParam("date") String date) {
        return ResponseEntity.ok(busService.searchBuses(from, to, date));
    }

    @GetMapping("/buses/{id}/seats")
    public ResponseEntity<SeatAvailabilityResponse> getSeatAvailability(
        @PathVariable("id") String id,
        @RequestParam("date") String date) {
        return ResponseEntity.ok(busService.getSeatAvailability(id, date));
    }

    @GetMapping("/cities")
    public ResponseEntity<List<String>> getAllCities() {
        return ResponseEntity.ok(busService.getAllCities());
    }
}

```

5.4.2 BookingController.java

```

@RestController
@RequestMapping("/api/bookings")
@RequiredArgsConstructor
public class BookingController {

```

```
private final BookingService bookingService;

@PostMapping
public ResponseEntity<BookingResponse> createBooking(
    @RequestBody BookingRequest request) {
    return ResponseEntity.ok(bookingService.createBooking(request));
}

@GetMapping("/{id}")
public ResponseEntity<BookingResponse> getBooking(
    @PathVariable("id") String id) {
    return ResponseEntity.ok(bookingService.getBooking(id));
}
}
```

5.5 Backend — Data Transfer Objects (DTOs)

DTOs define the API contract, ensuring internal entity structure is not exposed to clients.

5.5.1 BookingRequest.java

```
@Data @NoArgsConstructor @AllArgsConstructor
public class BookingRequest {
    private String busId;
    private String travelDate;
    private List<Integer> seatNumbers;
    private String passengerName;
    private String passengerEmail;
    private String passengerPhone;
}
```

5.5.2 BookingResponse.java

```
@Data @NoArgsConstructor @AllArgsConstructor
public class BookingResponse {
    private String bookingId;
    private String passengerName;
    private String passengerEmail;
    private String passengerPhone;
    private String busName;
    private String busNumber;
    private String origin;
    private String destination;
    private LocalDate travelDate;
    private List<Integer> seatNumbers;
    private BigDecimal totalFare;
    private String status;
    private LocalDateTime bookingTime;
}
```

5.5.3 BusSearchResponse.java

```
@Data @NoArgsConstructor @AllArgsConstructor
public class BusSearchResponse {
    private String busId;
    private String busName;
    private String busNumber;
    private String busType;
    private Integer totalSeats;
    private LocalTime departureTime;
    private LocalTime arrivalTime;
    private BigDecimal fare;
    private String origin;
}
```

```
private String destination;
private Integer availableSeats;
}
```

5.5.4 SeatAvailabilityResponse.java

```
@Data @NoArgsConstructor @AllArgsConstructor
public class SeatAvailabilityResponse {
    private String busId;
    private String busName;
    private Integer totalSeats;
    private List<Integer> bookedSeats;
    private BigDecimal fare;
}
```

5.6 Backend — Configuration & Exception Handling

5.6.1 CorsConfig.java

Enables Cross-Origin Resource Sharing for all origins, allowing the frontend (served from a different port or file) to communicate with the backend.

```
@Configuration
public class CorsConfig {
    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/**")
                    .allowedOrigins("*")
                    .allowedMethods("*")
                    .allowedHeaders("*");
            }
        };
    }
}
```

5.6.2 GlobalExceptionHandler.java

Centralised exception handling returns structured JSON error responses instead of stack traces.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
```



```

    public ResponseEntity<Map<String, String>> handleRuntimeException(
        RuntimeException ex) {
        return ResponseEntity.badRequest()
            .body(Map.of("error", ex.getMessage()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>> handleException(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "An unexpected error occurred: "
                + ex.getMessage()));
    }
}

```

5.6.3 application.properties

```

spring.data.mongodb.uri=mongodb://localhost:27017/bus_reservation
server.port=8080

```

5.7 Frontend — User Interface

The frontend consists of four static HTML pages that communicate with the backend via the **Fetch API**. Each page features a premium glassmorphism dark-mode design using Tailwind CSS with animated background orbs.

5.7.1 User Flow



5.7.2 Page Descriptions

Page	Features
index.html (Home)	• Hero section with "Travel Beyond Gravity" branding • Dynamic city dropdowns populated from <code>/api/cities</code> • Date picker with auto-set to tomorrow • Popular routes section with quick-search cards • Stats bar (10,000+ Bookings, 500+ Buses, 50+ Cities)
results.html (Search Results)	• Route & date header with "Modify Search" link • Loading skeleton animation • Bus cards showing name, type, number, departure/arrival times, fare, available seats • "View Seats" button linking to seat selection • Empty state with friendly message when no buses found
seats.html (Seat Selection)	• Interactive 2+2 seat grid with colour legend (Available / Booked / Selected) • Maximum 6 seats per booking with validation • Real-time booking summary panel (selected seats, total fare) • Passenger details form (name, email, phone) • Confirm Booking button with loading spinner
confirmation.html (Ticket)	• Animated success checkmark with pulse ring • Printable ticket card with booking ID, passenger info, route, bus, seats, fare • Print Ticket & Book Another Trip buttons

- Print-optimised CSS media query for clean printout

5.7.3 Design Highlights

- **Glassmorphism:** Semi-transparent cards with `backdrop-blur` and subtle borders.
- **Animated Orbs:** Three floating gradient circles with CSS `@keyframes drift` animation.
- **Gradient Accents:** Blue-to-purple gradients on buttons, headings, and text for a premium feel.
- **Responsive Design:** Tailwind's grid utilities ensure mobile-first, responsive layouts.
- **Graceful Fallback:** If the backend is unavailable, each page renders with realistic demo data.

6. User Interface Screenshots

The following describes each page of the BusVoyager application as seen in a web browser.

6.1 Home Page (index.html)

Description: The landing page features a dark-mode design with animated floating gradient orbs in the background. At the centre is the hero text "Travel Beyond Gravity" in large bold typography with a blue-to-purple gradient. Below the hero is a frosted-glass search card with three fields: "From" city dropdown, "To" city dropdown, and a date picker, followed by a gradient "Search Transport" button. Below the search form is a "Popular Routes" section showing clickable route cards, and a statistics bar displaying "10,000+ Bookings", "500+ Buses", "50+ Cities".

6.2 Search Results Page (results.html)

Description: This page shows a header bar with the selected route (e.g., "Delhi → Mumbai") and travel date. Below are bus result cards, each displaying the bus name, type badge (e.g., "AC"), bus number, departure and arrival times connected by a gradient line, fare in large bold text, available seats count in green, and a "View Seats" gradient button. If no buses are found, an empty state with a sad face icon and helpful text is shown.

6.3 Seat Selection Page (seats.html)

Description: The page is divided into two sections. The left (main) section contains the bus name with fare-per-seat, a colour legend (Available=grey, Booked=red, Selected=blue-purple gradient), and an interactive seat grid in a 2+aisle+2 layout. Users click seats to select/deselect them. Below the seat map, a passenger details form appears when seats are selected, with fields for full name, email, and phone. The right sidebar is a sticky "Booking Summary" card showing selected seat count, individual seat badges, total fare, and a "Confirm Booking" button.

6.4 Booking Confirmation Page (confirmation.html)

Description: A centred confirmation page with an animated green checkmark that pulses. The heading reads "Booking Confirmed!" in a green-to-blue gradient. Below is a ticket card showing: Booking ID (monospace font), CONFIRMED badge, passenger name and contact, journey route and date, bus name and number, seat number badges, and total amount paid. At the bottom are "Print Ticket" and "Book Another Trip" buttons.

7. Testing

The system was tested using a combination of API testing and manual UI testing.

7.1 API Testing (Manual / cURL)

#	Test Case	Method & Endpoint	Expected Result	Status
1	Search buses (valid route)	GET /api/buses/search?from=Delhi&to=Mumbai&date=2026-07-28	Returns list of buses with available seats	✓ Pass
2	Search buses (invalid route)	GET /api/buses/search?from=Delhi&to=Kolkata&date=2026-07-28	Returns empty list []	✓ Pass
3	Get seat availability	GET /api/buses/1/seats?date=2026-07-26	Returns booked seat numbers	✓ Pass
4	Get all cities	GET /api/cities	Returns sorted list of unique cities	✓ Pass
5	Create booking (valid)	POST /api/bookings	Returns BookingResponse with CONFIRMED status	✓ Pass
6	Create booking (duplicate seat)	POST /api/bookings (seat already booked)	Returns 400 with error message	✓ Pass
7	Create booking (invalid seat number)	POST /api/bookings (seat 45 on 40-seat bus)	Returns 400: "Invalid seat number: 45"	✓ Pass
8	Get booking by ID	GET /api/bookings/{id}	Returns full booking details	✓ Pass
9	Get booking (invalid ID)	GET /api/bookings/invalid123	Returns 400: "Booking not found"	✓ Pass
10	Bus not found	GET /api/buses/invalid/seats?date=2026-07-26	Returns 400: "Bus not found"	✓ Pass

7.2 Frontend Testing (Manual)

#	Test Case	Expected Behaviour	Status
1	City dropdowns load on page load	Both dropdowns populated with cities from API	✓ Pass
2	Search form validation	Cannot submit without selecting all three fields	✓ Pass
3	Seat selection — max 6 seats	Error message shown when trying to select 7th seat	✓ Pass
4	Booked seats are non-clickable	Red-tinted seats cannot be selected	✓ Pass
5	Fare calculation updates in real-time	Total fare = fare × number of selected seats	✓ Pass
6	Passenger form validation	"Please fill all passenger details" error if empty	✓ Pass
7	Booking confirmation displays ticket	All booking details rendered correctly	✓ Pass
8	Print ticket functionality	Browser print dialog opens with clean layout	✓ Pass
9	Responsive design (mobile)	All pages layout correctly on 375px width	✓ Pass
10	Fallback demo data when backend is down	Pages render with mock data gracefully	✓ Pass

8. Future Enhancements

While the current version provides complete bus search and booking functionality, the following enhancements are planned for future iterations:

#	Enhancement	Description
1	User Authentication	Implement JWT-based login/registration using Spring Security so passengers can track their booking history.
2	Payment Gateway	Integrate Razorpay or Stripe for secure online payments. Current system assumes offline payment.
3	Admin Dashboard	Build an admin panel for bus operators to manage buses, routes, view bookings, and generate revenue reports.
4	Booking Cancellation & Refunds	Allow passengers to cancel bookings with configurable refund policies based on cancellation timing.
5	Email / SMS Notifications	Send booking confirmations, reminders, and cancellation notices via email (JavaMailSender) or SMS (Twilio).
6	Seat Type Differentiation	Distinguish between window, aisle, and middle seats with different pricing tiers.
7	Live Bus Tracking	Integrate GPS tracking to show real-time bus locations on a map.
8	Rating & Review System	Allow passengers to rate and review bus services after travel.
9	Multi-Language Support	Add internationalisation (i18n) for Hindi, Tamil, and other regional languages.
10	Docker Deployment	Containerise the application with Docker Compose for MongoDB + Spring Boot for one-command deployment.

9. Conclusion

The **Bus Reservation System (BusVoyager)** successfully demonstrates the design and implementation of a full-stack web application using modern Java technologies. The project achieves its stated objectives:

- A clean, layered Spring Boot backend with RESTful APIs for bus search, seat availability, and booking management.
- MongoDB as a flexible NoSQL datastore with Spring Data repository abstractions.
- A visually premium frontend with interactive seat selection, real-time fare calculation, and printable booking tickets.
- Robust error handling and graceful fallback behaviour when the backend is unavailable.
- Comprehensive seed data covering multiple routes, buses, and sample bookings for immediate demonstration.

The project follows industry-standard practices including DTO pattern, global exception handling, CORS configuration, and responsive design. The modular architecture makes it easy to extend with authentication, payments, and admin features in future versions.

This project has been an excellent learning experience in full-stack development, covering backend API design, database modelling, frontend interactivity, and software engineering best practices.

10. References

1. Spring Boot Official Documentation — <https://spring.io/projects/spring-boot>
2. Spring Data MongoDB Reference — <https://spring.io/projects/spring-data-mongodb>
3. MongoDB Official Documentation — <https://www.mongodb.com/docs/>
4. Project Lombok — <https://projectlombok.org/> _
5. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
6. MDN Web Docs – Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
7. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
8. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
9. Baeldung — Spring Boot Tutorials — <https://www.baeldung.com/spring-boot>
0. Apache Maven Documentation — <https://maven.apache.org/guides/>

Appendix — Database Schema & Seed Data (schema.sql)

```
-- =====
-- Bus Reservation System - Database Schema & Seed Data
-- =====

CREATE DATABASE IF NOT EXISTS bus_reservation;
USE bus_reservation;

-- Drop existing tables
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS buses;
DROP TABLE IF EXISTS routes;
DROP TABLE IF EXISTS users;

-- Table: users
CREATE TABLE users (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(100) NOT NULL UNIQUE,
  phone       VARCHAR(15),
  password_hash VARCHAR(255),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table: routes
CREATE TABLE routes (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  origin       VARCHAR(100) NOT NULL,
  destination  VARCHAR(100) NOT NULL,
  distance_km  INT
);

-- Table: buses
CREATE TABLE buses (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  bus_name     VARCHAR(100) NOT NULL,
  bus_number   VARCHAR(20) NOT NULL,
  bus_type     ENUM('AC', 'NON_AC', 'SLEEPER', 'SEMI_SLEEPER') NOT NULL,
  total_seats  INT DEFAULT 40,
  route_id     BIGINT NOT NULL,
  departure_time TIME NOT NULL,
  arrival_time  TIME NOT NULL,
  fare         DECIMAL(10,2) NOT NULL,
  operating_days VARCHAR(100) DEFAULT 'MON,TUE,WED,THU,FRI,SAT,SUN',
  FOREIGN KEY (route_id) REFERENCES routes(id) ON DELETE CASCADE
);
```

```

-- Table: bookings
CREATE TABLE bookings (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  passenger_name VARCHAR(100) NOT NULL,
  passenger_email VARCHAR(100) NOT NULL,
  passenger_phone VARCHAR(15),
  bus_id      BIGINT NOT NULL,
  travel_date  DATE NOT NULL,
  seat_numbers TEXT NOT NULL,
  total_fare   DECIMAL(10,2) NOT NULL,
  status       ENUM('CONFIRMED','CANCELLED') DEFAULT 'CONFIRMED',
  booking_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (bus_id) REFERENCES buses(id) ON DELETE CASCADE
);

-- Seed Data: users
INSERT INTO users (name, email, phone) VALUES
('Rahul Sharma', 'rahul@example.com', '9876543210'),
('Priya Patel', 'priya@example.com', '9876543211'),
('Arjun Kumar', 'arjun@example.com', '9876543212');

-- Seed Data: routes (8 routes, 4 bidirectional pairs)
INSERT INTO routes (id, origin, destination, distance_km) VALUES
(1, 'Delhi', 'Mumbai', 1400),
(2, 'Mumbai', 'Delhi', 1400),
(3, 'Bangalore', 'Chennai', 350),
(4, 'Chennai', 'Bangalore', 350),
(5, 'Hyderabad', 'Pune', 560),
(6, 'Pune', 'Hyderabad', 560),
(7, 'Jaipur', 'Delhi', 280),
(8, 'Delhi', 'Jaipur', 280);

-- Seed Data: 12 buses across different routes
INSERT INTO buses (id, bus_name, bus_number, bus_type, total_seats,
  route_id, departure_time, arrival_time, fare, operating_days) VALUES
(1, 'Royal Cruiser', 'KA-01-AB-1234', 'AC', 40, 1,
  '18:00:00', '06:00:00', 1299.00, 'MON,TUE,WED,THU,FRI,SAT,SUN'),
(2, 'Star Express', 'KA-01-CD-5678', 'SLEEPER', 40, 1,
  '20:00:00', '08:00:00', 899.00, 'MON,TUE,WED,THU,FRI,SAT,SUN'),
-- ... (10 more buses omitted for brevity)
(12, 'Pink City Express', 'DL-01-MN-4433', 'AC', 40, 8,
  '16:00:00', '21:00:00', 399.00, 'MON,TUE,WED,THU,FRI,SAT,SUN');

-- Seed Data: sample bookings
INSERT INTO bookings (passenger_name, passenger_email, passenger_phone,
  bus_id, travel_date, seat_numbers, total_fare, status) VALUES
('Rahul Sharma', 'rahul@example.com', '9876543210',
  1, '2026-07-26', '[1,2,5,6]', 5196.00, 'CONFIRMED'),
('Priya Patel', 'priya@example.com', '9876543211',
  1, '2026-07-26', '[11,12,15]', 3897.00, 'CONFIRMED'),

```

```
('Arjun Kumar', 'arjun@example.com', '9876543212',  
6, '2026-07-26', '[3,4,7,8]', 1996.00, 'CONFIRMED');
```

Appendix B — Maven Configuration (pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.1</version>
  </parent>
  <groupId>com.busreserve</groupId>
  <artifactId>bus-reservation</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>bus-reservation</name>
  <description>Bus Reservation System Backend</description>
  <properties>
    <java.version>25</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <configuration>
          <release>25</release>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

```
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

— End of Report —